

* Time Complexity & Space complexity

Asymptotic Notation

- 1) Big oh (O) - Defines the upper bound of the running time of an algorithm as the input size n increases.

~~$f(n) = O(n^2)$~~
worst case $\rightarrow O(n^2)$

- 2) Big Omega (Ω) - It describes the lower bound of the running time of an algorithm. It tells us the best-case time complexity.

~~$f(n) = \Omega(n \log n)$~~
Best case $\rightarrow \Omega(n \log n)$

- 3) Big theta (Θ) - It describes the average rate of time of running an algorithm. It tells us the Average-case time complexity.

$f(n) = \Theta(n)$
Average case $\rightarrow \Omega(n)$

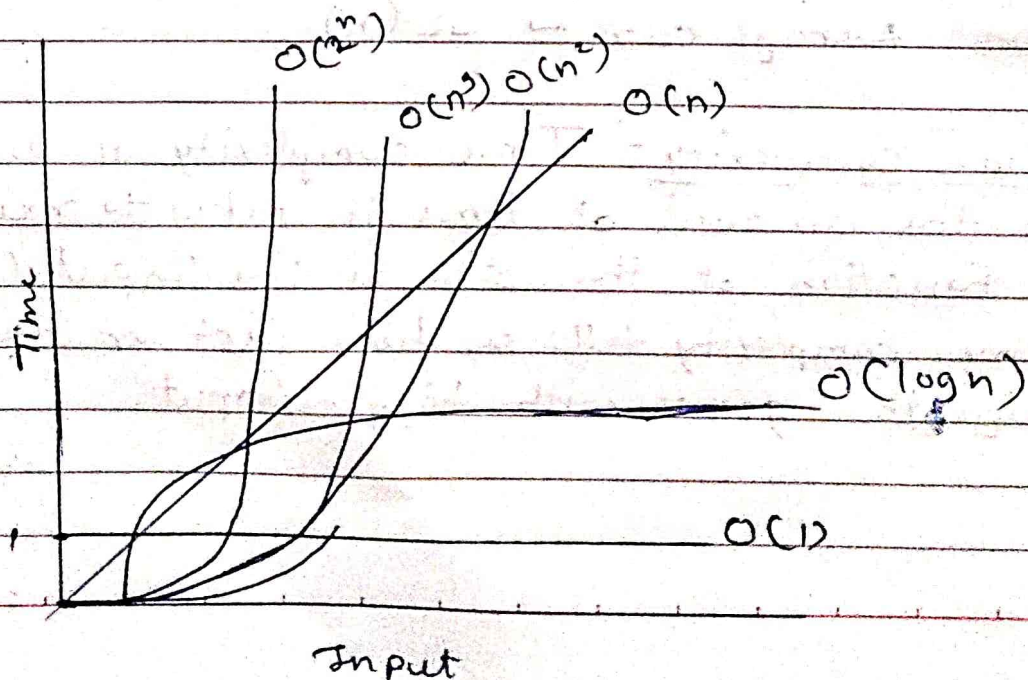
Time complexity - Time complexity of an algorithm is the amount of time it takes to run as a function of the size of the input (n).

Time complexity tells us how fast or slow an algorithm grows with bigger input.

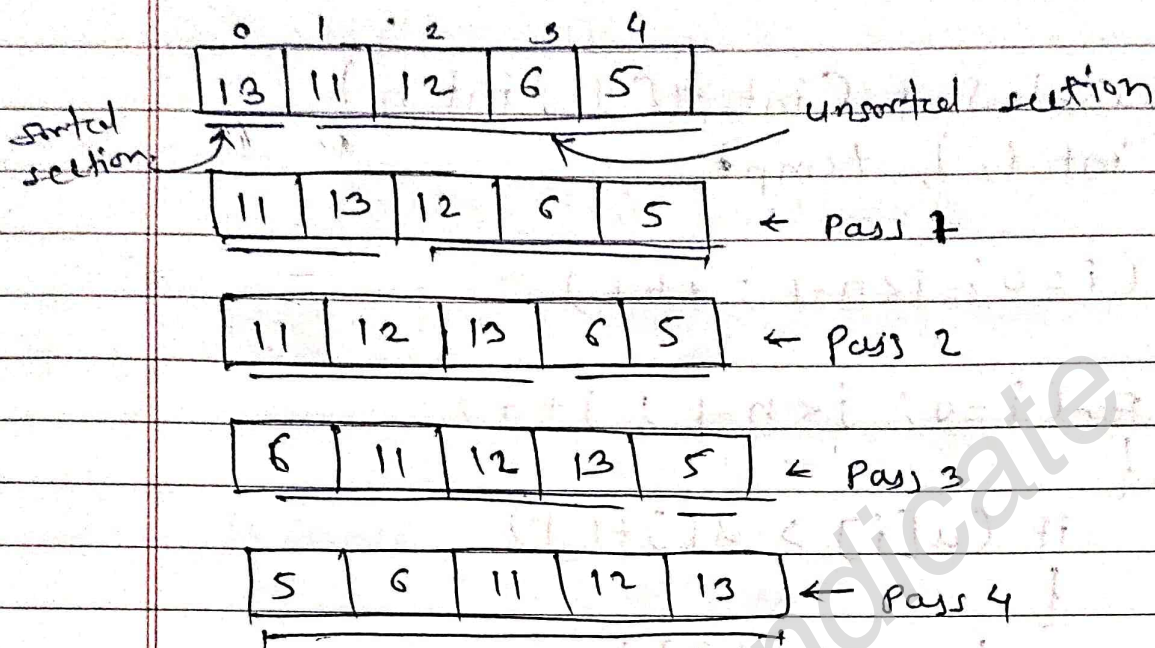
Space Complexity - Space complexity of an algorithm is the amount of memory (space) it needs to run, as a function of the input size (n).

Q Difference between Time complexity and space complexity

Time complexity.	Space complexity.
1) Measures the amount of time takes to run an algorithm.	Measures the amount of memory takes to run an algorithm.
2) It Depends on Number of operations (loops, recursion, comparisons)	It Depends on variables, input size, recursion stack, data structures.
3) Uses to minimize execution time	Uses to minimize memory usage.
4) Notation - Big-O, Big- Ω , Big- Θ ($O(n^2), \Theta(n)$)	Notation - Big-O, Big- Ω , Big- Θ ($O(1), \Theta(n)$).



* Insertion Sort



```

void insertionSort(int a[], int n)
{
    int i, j, num;
    for (i = 1; i < n; i++)
    {
        num = a[i];
        for (j = i - 1; j >= 0; j--)
        {
            if (a[j] > num)
            {
                a[j + 1] = a[j];
            }
            else
            {
                break;
            }
        }
        a[j + 1] = num;
    }
}

```


number of comparison for pass 1 $\rightarrow 1$

Pass 2 $\rightarrow 2$

Pass 3 $\rightarrow 3$

Pass $n-1 \rightarrow n-1$

$$f(n) = 1 + 2 + 3 + \dots + n-1$$

$$f(n) = \frac{n(n-1)}{2}$$

$$f(n) = \frac{n^2 - n}{2}$$

$$f(n) = O(n^2)$$

* Selection Sort

14	13	10	25	7
----	----	----	----	---

Selection Sort repeatedly selects the minimum element from the unsorted portion and swaps it into next position.

① For each position find the smallest element and swap it with a $[i]$.

② Perform the same number of comparisons regardless of input or not is ordered.

1	14	13	10	25	7		7	10	13	25	14	-P ₃
---	----	----	----	----	---	--	---	----	----	----	----	-----------------

Pass 1	7	13	10	25	14		7	10	13	14	25	-P ₂
--------	---	----	----	----	----	--	---	----	----	----	----	-----------------

Pass 2	7	10	13	25	14
--------	---	----	----	----	----

Algorithm

Void SelectionSort (int a[], int n)

```
{
    int i, j, small, b;
    for (i = 0; i < n-1; i++)
    {
```

```
        small = a[i]
```

```
        b = i
```

```
        for (j = i+1; j < n; j++)
        {
```

```
            if (small > a[j])
```

```
            {
```

```
                small = a[j];
```

```
                b = j;
            }
```

```
        }
```

```
    }
```

```
    a[b] = a[i];
```

```
    a[i] = small;
```

```
}
```

```
}
```

Time & space complexity

Best / worst / Average Time : $O(n^2)$

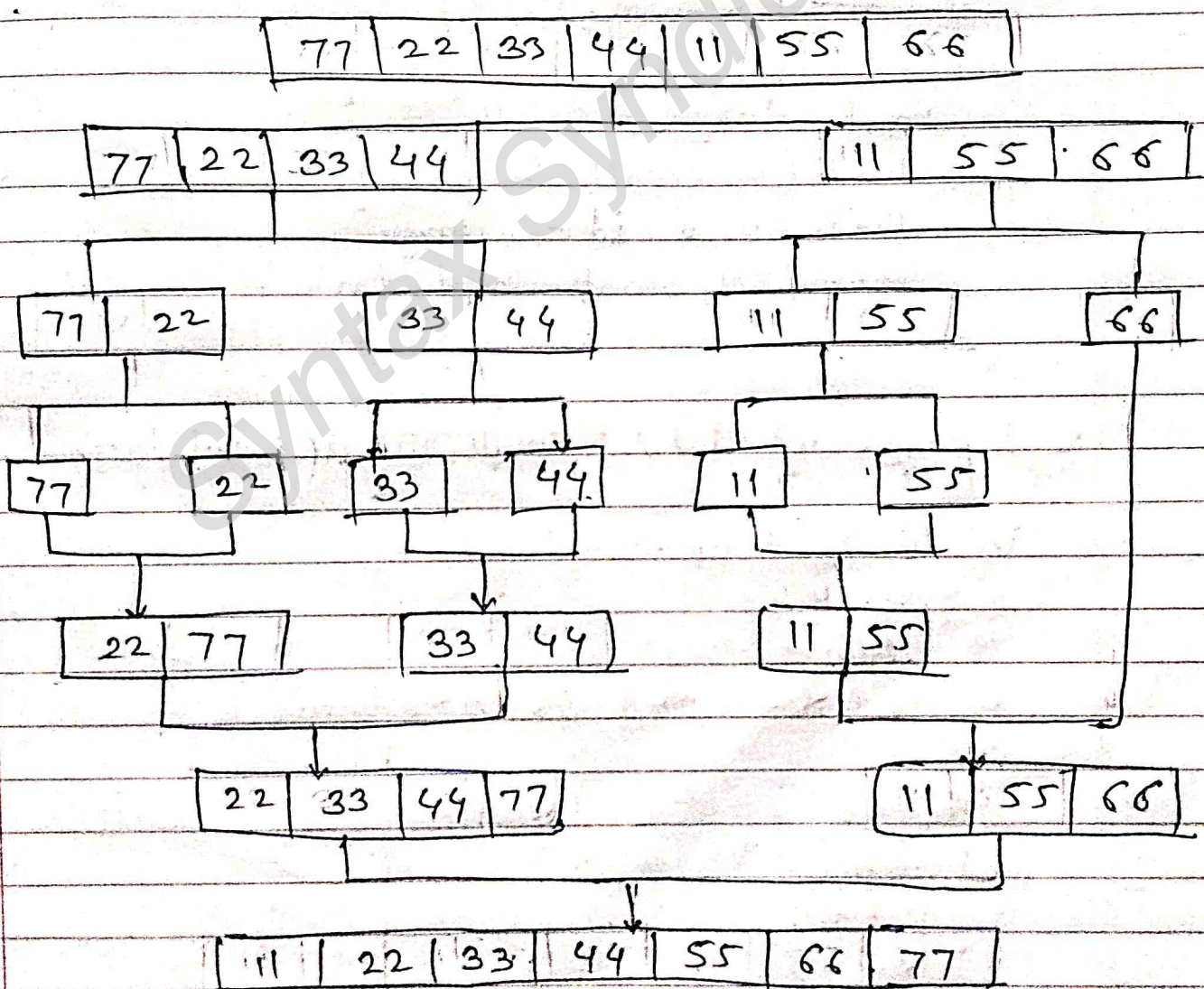
Space : $O(1)$ = in place.

*

Merge Sort

Merge sort sorts the list using divide and conquer strategy, Merge sort work in three stages

- Divide: Recursively divide the single list into two sublist until each sublist contain 1 element
- Conquer: Sort both sublist of parent list. List of 1 element is sorted
- Combine: Recursively combine sorted sublists into the list of size n is produced.



Time complexity

Best / worst / Average = $\Theta(n \log n)$

* Quick sort

Quick sort is a divide-and-conquer sorting algorithm that partitions the array around a pivot, then recursively sorts the left and right subarrays.

- Choose a pivot, place it in correct sorted position by moving smaller elements to its left and large to its right.
- Recursively apply to subarrays on both side of pivot.

A

10	80	30	90	40	50	70
----	----	----	----	----	----	----

Pivot = 70, after partition

10	30	40	50	70	90	80
----	----	----	----	----	----	----

Sort left

10	30	40	50
----	----	----	----

Sort right

90	80
----	----

Sorted array

10	30	40	50	70	80	90
----	----	----	----	----	----	----

Algorithm

```
void quickSort (int a[], int low, int high) {  
    if (low < high) {  
        int p = partition (a, low, high);  
        quicksort (a, low, p-1);  
        quicksort (a, p+1, high);  
    }  
}
```


Time & Space Complexity

Best case : $O(n \log n)$

Worst/Average case = ~~$O(\log n)$~~ , $O(n^2)$

Space : $O(\log n)$
: $O(n)$

* Binary Search:

Binary Search finds a target in a sorted array by repeatedly dividing the search range in half and comparing the middle element to the target.

- Check middle element:

- IF equal $\rightarrow$ Found
- IF large $\rightarrow$ Search right half
- IF small $\rightarrow$ Search left half

- Repeat until Found or range Empty.
works only on sorted arrays.

10	20	30	40	50	60
0	1	2	3	4	5

Search = 40

• mid = 2 (30) < 40 $\rightarrow$ search right
l = 3, h = 5

• mid = 4 (50) > 40 $\rightarrow$ search left
l = 3, h = 3

• mid = 3 (40) = 40 Found at index 3.

Algorithm

```
int binarySearch (int a[], int x, int n) {  
    int l = 0, h = n - 1;  
    while (l <= h) {  
        int m = (l + h) / 2;  
        if (a[m] == x)  
            return m;  
        else if (a[m] < x)  
            l = m + 1;  
        else  
            h = m - 1;  
    }  
    return -1;  
}
```

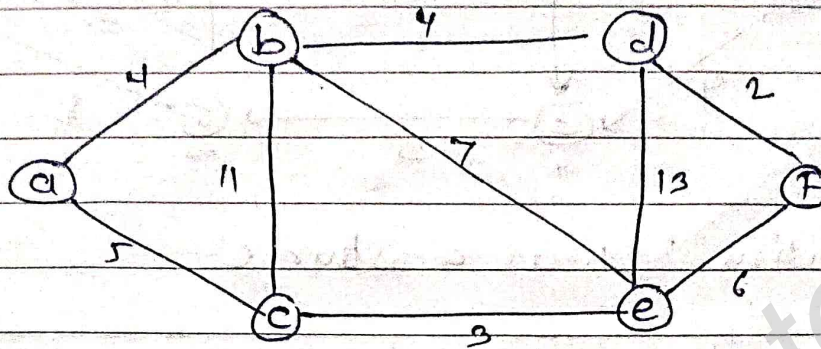
Space & Time complexity.

Best case: $\Theta(1)$ - target is middle element
worst case: $\Theta(\log n)$ element not in array.

Space = $O(1)$ For iterative version,
= $O(\log n)$ For recursive version.

* Dijkstra's Algorithm

①



Initialization : a as a source

Selected vertex	a	b	c	d	e	f
a	0	∞	∞	∞	∞	∞
b	0	4	5	∞	∞	∞
c	0	4	5	8	∞	∞
d	0	4	5	8	8	∞
e	0	4	5	8	8	10
f	0	4	5	8	8	10

Shortest distance from a

$$a \rightarrow a = 0$$

$$a \rightarrow b = 4$$

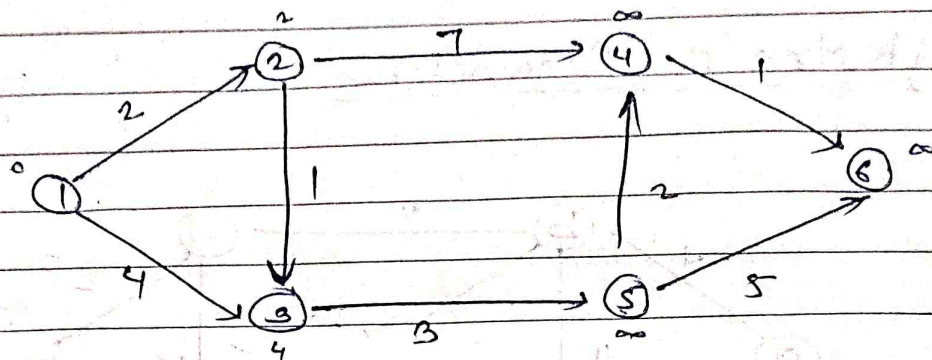
$$a \rightarrow c = 5$$

$$a \rightarrow d = 8$$

$$a \rightarrow e = 8$$

$$a \rightarrow f = 10$$

②



Initialization : 1 as a source

selected vertex	1	2	3	4	5	6
1	0	∞	∞	∞	∞	∞
2	0	2	4	∞	∞	∞
3	0	2	3	9	8	8
5	0	2	3	9	6	∞
4	0	2	3	8	6	11
6	0	2	3	8	6	9

Shortest distance from 1

$$1 \rightarrow 1 = 0$$

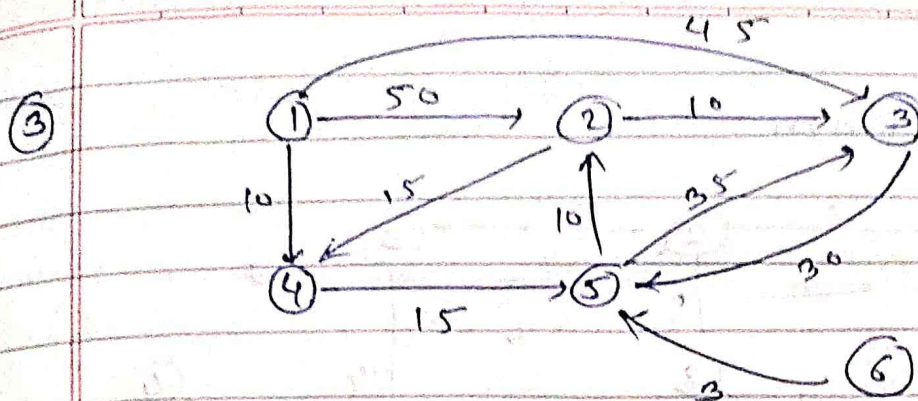
$$1 \rightarrow 2 = 2$$

$$1 \rightarrow 3 = 3$$

$$1 \rightarrow 4 = 8$$

$$1 \rightarrow 5 = 6$$

$$1 \rightarrow 6 = 9$$



Initialization : 1 as a source

Selected vertex	1	2	3	4	5	6
1	0	∞	∞	∞	∞	∞
4	0	50	45	10	∞	∞
5	0	50	45	10	25	∞
2	0	35	45	10	25	∞
3	0	35	45	10	25	∞

shortest distance from 1

$$1 \rightarrow 1 = 0$$

$$1 \rightarrow 2 = 35$$

$$1 \rightarrow 3 = 45$$

$$1 \rightarrow 4 = 10$$

$$1 \rightarrow 5 = 25$$

$$1 \rightarrow 6 = \infty$$

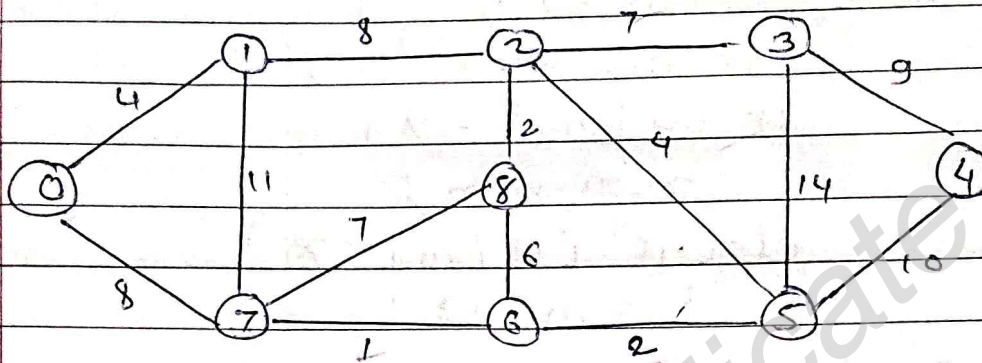


~~Dijkstra's~~



Kruskal's Algorithm

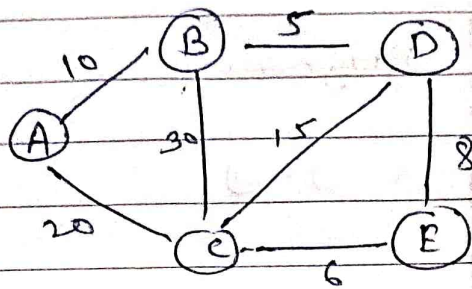
1)



Selected edge	Estimate cost	Total cost	Diagram
7-6	1	1	7-6
6-5	2	3	7-6-5
2-8	2	5	7-6-5 2-8
0-1	4	9	7-6-5 2-8 0-1
2-5	4	13	7-6-5 2-8 0-1 2-5
2-3	7	20	7-6-5 2-8 0-1 2-5 2-3
1-2	8	28	7-6-5 2-8 0-1 2-5 2-3 1-2
3-4	9	37	7-6-5 2-8 0-1 2-5 2-3 1-2 3-4

$\therefore$ Total cost of Kruskal's Algorithm is 37

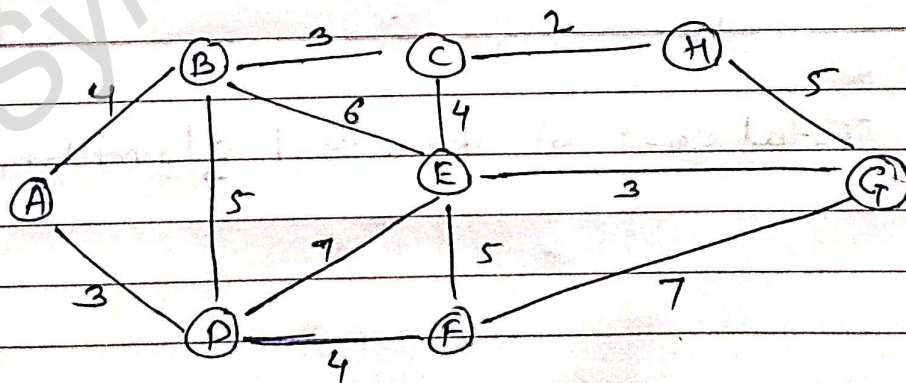
2)



Selected Edge	Estimate cost	Total cost	Diagram	Node cover.
B-D	5	5		B, D
C-E	6	11		B, D, C, E
D-E	8	19		B, D, C, E
A-B	10	29		B, D, C, E, A

$\therefore$ Total cost of Kruskal's algorithm is 29

3)

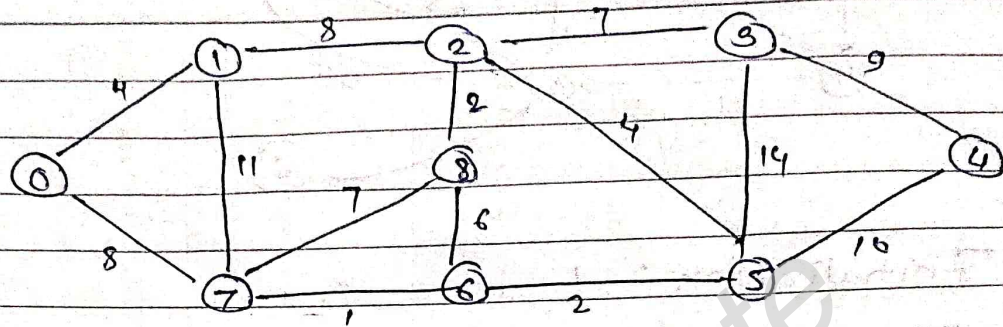


Selected edge	Estimate cost	Total cost	Diagram	Node cover
C-H	2	2	C-H	C, H
A-D	3	5	C-H A-D	C, H, A, D
B-C	3	8	C-H A-D B-C	C, H, A, D, B
E-G	3	11	C-H A-D B-C E-G	C, H, A, D, B, E, G
C-E	4	15	C-H A-D B-C E-G C-E	C, H, A, D, B, E, G
A-B	4	19	C-H A-D B-C E-G C-E A-B	C, H, A, D, B, E, G
D-F	4	23	C-H A-D B-C E-G C-E A-B D-F	C, H, A, D, B, E, G

∴ Total cost of Kruskal algorithm 23

* Prims Algorithm

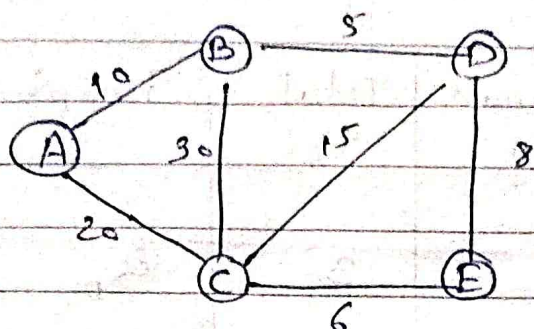
1)



Edge Consider	Selected edge	Estimate cost	Total cost	Graph	Node covered
0-1, 0-7	0-1	4	4	0-1	0, 1
0-7, 1-2, 1-7	0-7	8	12	0-1-7	0, 1, 7
1-2, 1-7, 7-8 7-6	7-6	1	13	0-1-7-6	0, 1, 7, 6
1-2, 1-7, 7-8 6-8, 6-5	6-5	2	15	0-1-7-6-5	0, 1, 7, 6, 5
1-2, 1-7, 7-8 6-8, 5-2, 5-3, 5-4	5-2	2	19	0-1-7-6-5-2	0, 1, 7, 6, 5, 2
1-2, 1-7, 7-8 6-8, 5-3, 5-4 2-8, 2-3	2-8	2	21	0-1-7-6-5-2-8	0, 1, 7, 6, 5, 2, 8
1-2, 1-7, 7-8 6-8, 5-3, 5-4 2-3,	2-3	7	28	0-1-7-6-5-2-8-3	0, 1, 7, 6, 5, 2, 8, 3
1-2, 1-7, 7-8 6-8, 5-3, 5-4 3-4	3-4	9	37	0-1-7-6-5-2-8-3-4	0, 1, 7, 6, 5, 2, 8, 3, 4

∴ Total cost of Prims Algorithm is 37.

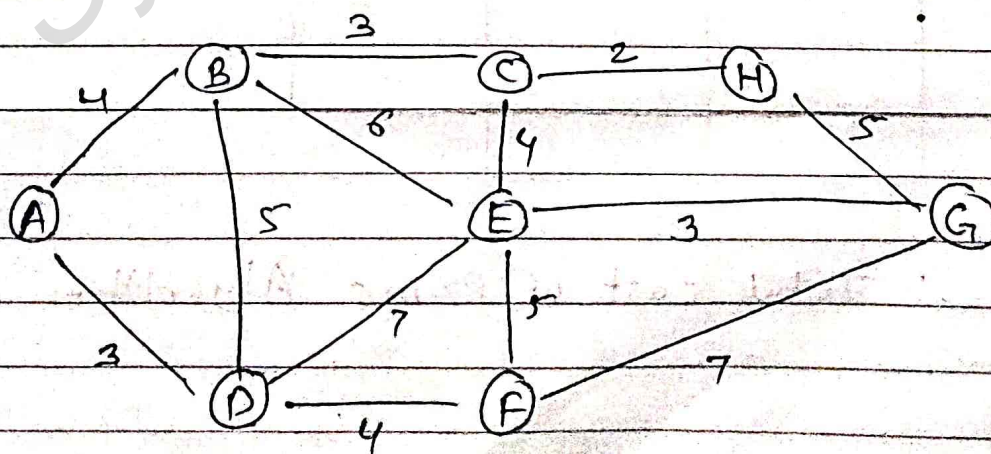
2)

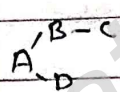
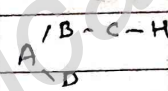
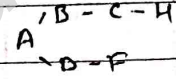
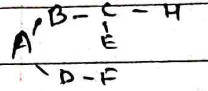
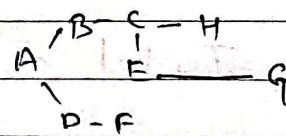


Edge consider	Selected edge	Estimate cost	Total cost	Graph	Node covered
A-B, A-C	A-B	10	10	A-B	A, B
A-C, B-C, B-D	B-D	5	15	A-B-D	A, B, D
A-C, B-C, D-E, D-C	D-E	8	23	A-B-D-E	A, B, D, E
A-C, B-C, D-E, E-C	E-C	6	29	A-B-D-E-C	A, B, D, E, C

$\therefore$ Total cost of Prim's Algorithm is 29

3)



Edge consider	selected Edge	Estimate cost	Total cost	Graph	Node cover
A-B, A-D	A-D	3	3		A, D
A-B, D-F, D-B, D-E	A-B	4	7		A, D, B
D-F, D-B, D-E, B-C, B-E	B-C	3	10		A, D, B, C
D-F, D-B, D-E, B-E, C-H, C-E	C-H	2	12		A, D, B, C, H
D-F, D-B, D-E, B-E, C-E, H-G	D-F	4	16		A, D, B, C, H, F
D-B, D-E, B-E, C-E, H-G, F-G, F-E	C-E	4	20		A, D, B, C, H, F, E
D-B, D-E, B-E, C-E, H-G, F-G, F-E, E-G	E-G	3	23		A, D, B, C, H, F, E, G

∴ Total cost of Prim's Algorithm is 23

* Fractional Knapsack Problem

$n = 7, m = 15$ (capacity)

$(P_1, P_2, P_3, P_4, P_5, P_6, P_7) = (10, 5, 15, 7, 6, 18, 3)$

$(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1)$

1) Descending Profit.

Object	Profit (P_i)	Weight	Remain weight
-	-	-	15
6	18	4	$15 - 4 = 11$
3	15	5	$11 - 5 = 6$
1	10	2	$6 - 2 = 4$
4	7	4	$4 - 4 = 0$

9

$\therefore$ Total profit is 47

2) Ascending weight

Object	Profit (P _i)	weight	Remain weight
-	-	-	15
5	6	1	$15 - 1 = 14$
7	3	1	$14 - 1 = 13$
1	10	2	$13 - 2 = 11$
2	5	3	$11 - 3 = 8$
6	18	4	$8 - 4 = 4$
3	$15 \times \frac{4}{5} = 12$	4	$4 - 4 = 0$

∴ Total profit is 54

* Greedy Approach

object	1	2	3	4	5	6	7
P _i	10	5	15	7	6	18	3
w _i	2	3	5	7	1	4	1
P _i /w _i	5	1.6	3	1	6	4.5	3

Object	Profit (P_i)	weight (w_i)	Remain weight
-	-	-	15
5	8	1	14
1	10	2	12
6	18	4	8
3	15	5	3
7	3	1	2
2	$5 \times \frac{2}{3} = 3.33$	2	0

$\therefore$ Total Profit is 55.33

2)

Object	1	2	3	4	5	6
P_i	5	10	15	7	8	9
w_i	1	3	5	4	1	3
P_i/w_i	5	3.33	3	1.75	8	3

Object	Profit (P_i)	weight (w_i)	Remain weight
—	—	—	15
5	8	1	14
1	5	1	13
2	10	3	10
3	15	5	5
6	9	3	2
4	$7 \times \frac{2}{4} = 3.5$	4	0

∴ Total profit is 50.5